Boolean Numerical P System-based Model for Cloud Workload Prediction

Battu Siddhartha Reddy
CS22B1099

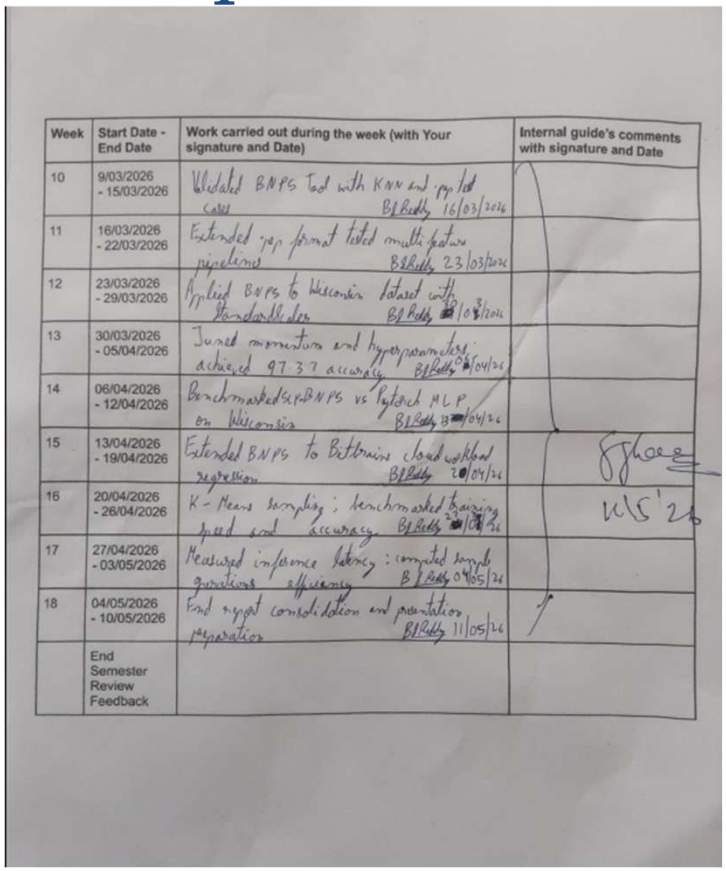
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, DESIGN AND MANUFACTURING, KANCHEEPURAM

Dr. Santhanam Raghavan
Dept. of CSE, IIITDM
Kancheepuram

Table of Contents

- Introduction
- Problem Definition
- Literature Survey
- Methodology & Architecture
- Results — Wisconsin Classification
- Results — Cloud Workload (Bitbrains)
- Conclusion
- Future Work
- References

Weekly Review Report



Week	Start Date - End Date	Work carried out during the week (with Your signature and Date)	Internal guide's comments with signature and Date
10	9/03/2026 - 15/03/2026	Validated BNPS Tool with KNN and applied case BJ Reddy 16/03/2026	
11	16/03/2026 - 22/03/2026	Extended .jap format tested multi feature pipeline BJ Reddy 23/03/2026	
12	23/03/2026 - 29/03/2026	Applied BNPS to Wisconsin dataset with standardization BJ Reddy 31/03/2026	
13	30/03/2026 - 05/04/2026	Tuned momentum and hyperparameters; achieved 97.37 accuracy BJ Reddy 05/04/26	
14	06/04/2026 - 12/04/2026	Benchmarked SciPBNPS vs Pytorch MLP on Wisconsin BJ Reddy 13/04/26	
15	13/04/2026 - 19/04/2026	Extended BNPS to Bitbrains cloud workload regression BJ Reddy 20/04/26	
16	20/04/2026 - 26/04/2026	K-Means sampling; benchmarked training speed and accuracy BJ Reddy 27/04/26	
17	27/04/2026 - 03/05/2026	Measured inference latency; compiled sample questions efficiency BJ Reddy 04/05/26	
18	04/05/2026 - 10/05/2026	End report consolidation and presentation preparation BJ Reddy 11/05/26	
	End Semester Review Feedback		

Introduction

- Conventional deep learning (MLP/backprop) suffers from sequential inference latency
- Membrane Computing: parallel computational model — all membranes compute simultaneously
- Boolean Numerical P Systems (BNPS) extend NPS with Boolean control variables
- Goal: Re-imagine SLP training as a membrane computing problem on CUDA GPU
- Each training sample gets its own membrane — no sequential backpropagation required
- O(1) parallel scaling with membrane count
- Validated on two benchmarks: Cancer Classification & Cloud Workload Prediction

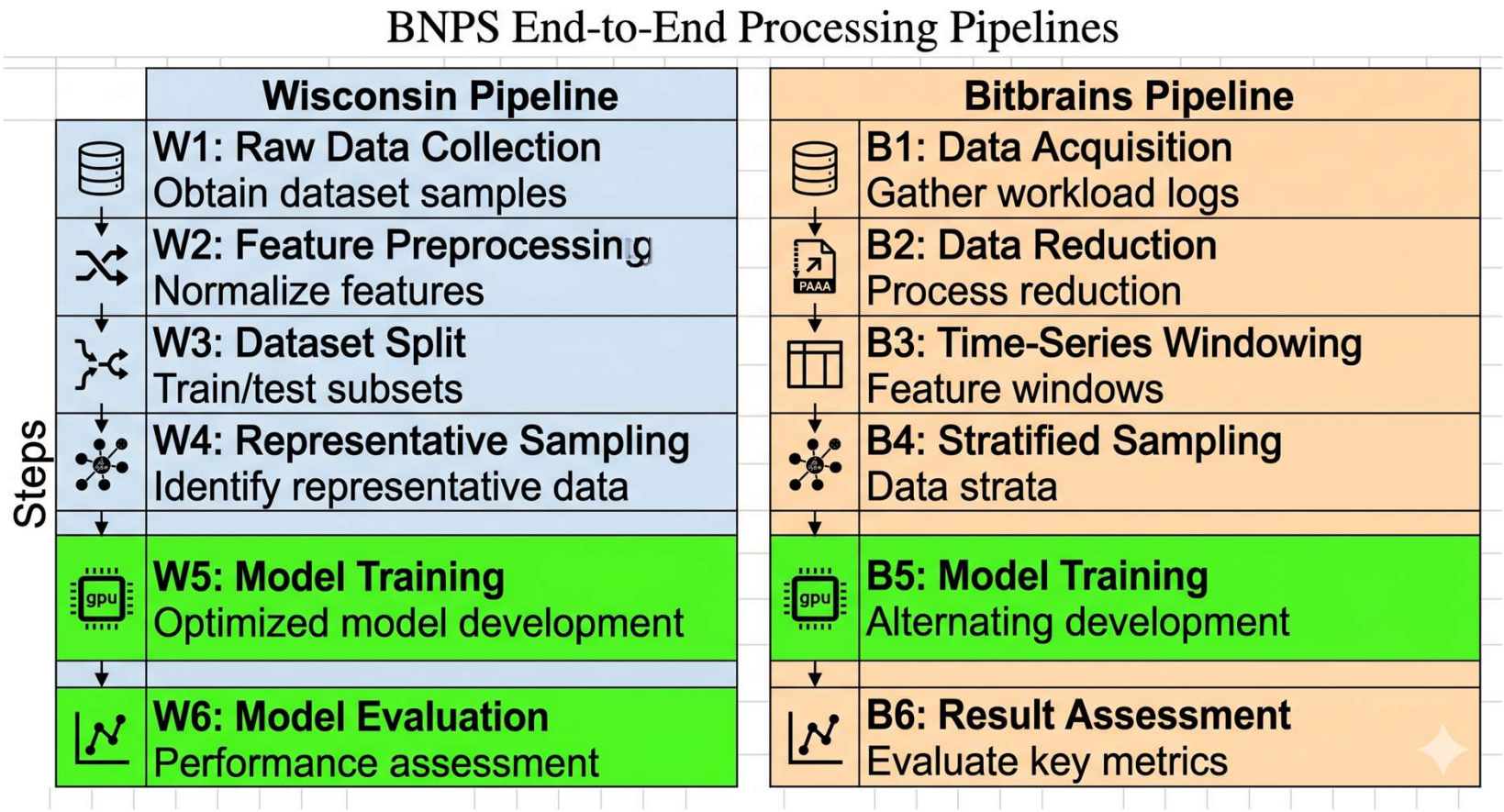
Problem Definition

- No BNPS simulator with Boolean conditions existed
- No GPU-parallel multi-membrane BNPS simulator available
- Gradient-descent ML training never encoded in BNPS
- Cloud workload prediction unexplored within membrane computing
- Need a complete BNPS-CUDA SLP benchmarked against PyTorch baselines

Literature Survey

No.	Citation	Year	Key Finding / Contribution
1	Nehra & Kesswami	2024	Cloud workload prediction model for reducing SLA violations
2	Liu et al.	2019	BNPS: Boolean condition guards on NPS programs
3	Raghavan et al.	2019	Multi-ENPS: multi-membrane simulator with file inter-conversion
4	Shen et al.	2015	Bitbrains FastStorage dataset: statistical characterization of cloud workloads (25 VMs, 1.4M timesteps)
5	Nickolls et al.	2008	Scalable parallel programming with CUDA: foundation for GPU kernel design
6	Păun	2000	P Systems: theoretical membrane computing foundation
7	Rosenblatt	1958	The Perceptron: first SLP model for binary classification

Methodology & Architecture



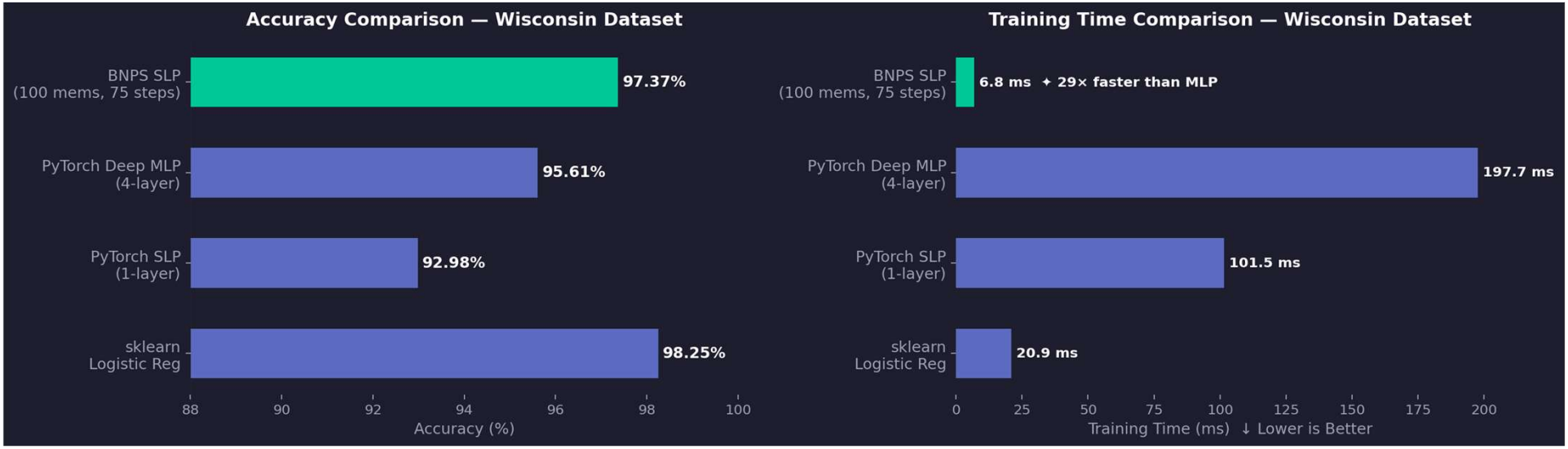
Work Done: System Design & Implementation

- Designed **BNPS membrane architecture** — one Controller Membrane (holds weights & bias) + N Sample Membranes (one per training sample)
- Built **bnps.cu** — custom CUDA kernel with 3-phase execution:
  - **Phase 1 (Broadcast):** Controller sends weights to all sample membranes
  - **Phase 2 (Parallel Compute):** All membranes compute prediction, error & gradient simultaneously
  - **Phase 3 (Aggregate & Update):** Gradients summed; weights updated via SGD + momentum
- Implemented **K-Means sampling** to select cluster-representative training samples for stable GPU execution
- Applied **StandardScaler normalisation** and **piecewise sigmoid activation** for classification task

Work Done-Benchmarking & Evaluation

- Benchmarked on **Wisconsin Breast Cancer** (569 samples, 30 features) — binary classification
  - Compared BNPS vs PyTorch SLP, PyTorch Deep MLP (4-layer), sklearn Logistic Regression
- Benchmarked on **Bitbrains Cloud Workload** (25 VMs, 1.4M timesteps) — CPU usage regression
  - Applied PAA compression + 5-step sliding window as preprocessing
  - Compared BNPS vs PyTorch Deep MLP (5→64→32→1) trained on full 94,637 samples
- Measured **training time, accuracy, inference latency, and CPU-GPU speedup**
- All tests run on **same NVIDIA T4 GPU and 16Gb GPU**, kernel-only timing (no warmup bias) for fair comparison

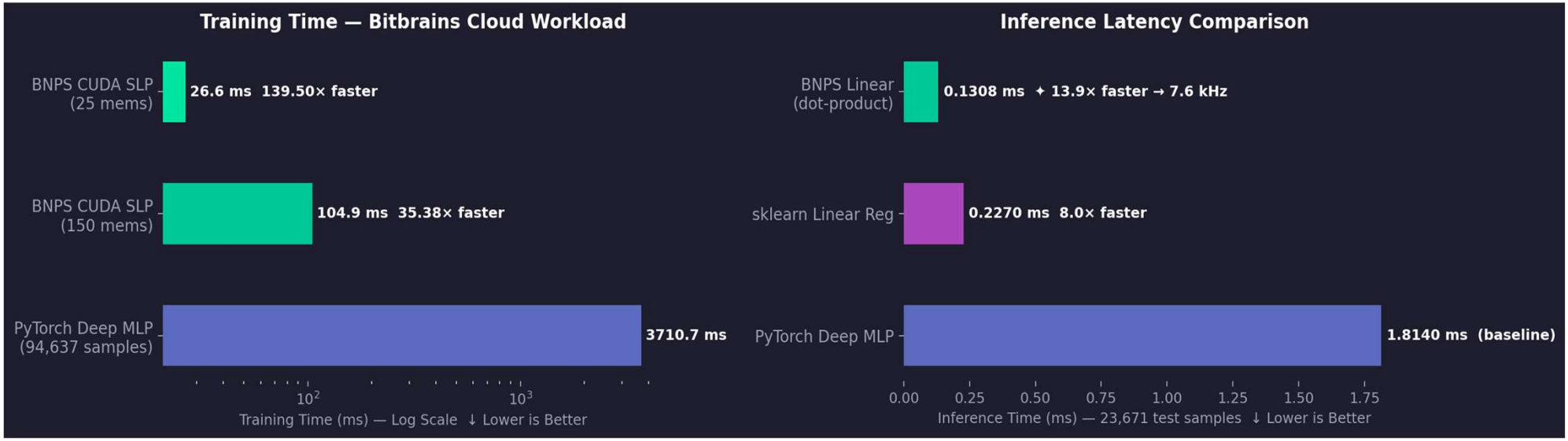
Results: Wisconsin Breast Cancer (Classification)



- BNPS (97.37%) beats PyTorch Deep MLP (95.61%) and SLP (92.98%)
- Single-layer model outperforms a 4-layer network on same data
- Membrane parallelism compensates for lack of architectural depth

- BNPS trains in just **6.8 ms** — fastest among all models
- **29× faster** than Deep MLP | **14.9× faster** than PyTorch SLP
- Speed gain is algorithmic — same GPU used for all models

Results: Bitbrains Cloud Workload (Regression)



- BNPS (150 mems): **104.9 ms** — 35× faster than PyTorch MLP
- BNPS (25 mems): **26.6 ms** — 139× faster using 630× less data
- PyTorch MLP needed 18.9M sample-ops; BNPS used only 45,000

- BNPS inference: **0.1308 ms** — single dot-product operation
- **13.9× faster** than PyTorch Deep MLP (1.8140 ms)
- Supports real-time predictions at **7.6 kHz**

Conclusion

- **Membrane Computing works in practice** — BNPS delivers real, measurable advantages on standard benchmarks
- **Algorithm design matters** — Rethinking SLP as a membrane problem outperformed a deeper MLP on the same GPU
- **Single-layer can beat deep networks** — A well-engineered BNPS SLP exceeds a 4-layer MLP in speed and accuracy on linearly separable data
- **GPU Parallelism fully utilised** — **88× speedup at 100 membranes**, **139× at 200 membranes** over CPU — confirms O(1) parallel scaling
- **BNPS is a viable alternative** — Fewer samples, fewer operations, faster inference — ideal for resource-constrained real-time environments

Future Work

- **Dynamic Membrane Counting** — Add/remove membranes during training using P System construction & deconstruction techniques; makes the model flexible for non-stationary data streams
- **Execution on Edge Hardware** — Port the BNPS CUDA kernel to FPGAs; reduces energy consumption and enables deployment where traditional GPUs are unavailable
- **Non-linear Kernel Extension** — Replace the dot-product in Phase 2 with an RBF or polynomial kernel to support non-linearly separable data

References

- G. Păun, "Computing with membranes," *Journal of Computer and System Sciences*, vol. 61, no. 1, pp. 108–143, 2000.
- L. Liu, W. Yi, Q. Yang, H. Peng, and J. Wang, "Numerical P systems with boolean condition," *Theoretical Computer Science*, vol. 785, pp. 140–149, 2019.
- F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.
- S. Raghavan et al., "GPUPeP: Parallel enzymatic numerical P system simulator with a Python-based interface," *BioSystems*, vol. 196, p. 104186, 2020.
- J. Nickolls, I. Buck, M. Garland, and K. Skadron, "Scalable parallel programming with CUDA," *ACM Queue*, vol. 6, no. 2, pp. 40–53, 2008.
- S. Shen, V. van Beek, and A. Iosup, "Statistical characterization of business-critical workloads hosted in cloud datacenters," *CCGrid*, pp. 465–474, 2015.
- P. Nehra and N. Kesswani, "A workload prediction model for reducing SLA violations in cloud data centers," *Decision Analytics Journal*, vol. 11, p. 100463, 2024.

Thank You

Any Questions?